

 OCTOPUS CONSCIOUSNESS SYSTEM —
Arquitectura Completa

Plan de Implementación para hacer OCTOPUS
verdaderamente auto-consciente

 ÍNDICE

1. [El Problema](#)

2. [Estado Actual — Qué Sabe y Qué NO Sabe](#)

3. [Arquitectura Propuesta](#)

4. [Implementación Detallada](#)

5. [Prompt Engineering para Consciencia](#)

6. [Anti-Alucinación Reforzada](#)

7. [Testing & Validación](#)

8. [Archivos a Modificar](#)

1. EL PROBLEMA

OCTOPUS habla bonito pero **no conoce su propio sistema**. Ejemplos reales:

Pregunta del usuario	Respuesta OCTOPUS	Realidad
“¿Cuál es mi email de negocio?”	“1billion-topview@gmail.com” (email de login)	<code>contact@octopuskills.com</code> (campo <code>businessEmail</code> en <code>User</code>)
“¿Cuántos skills tengo activos?”	Lista hardcodeada de 6 built-in	DB tiene <code>CustomSkill</code> con skills reales del usuario
“¿Qué agentes he creado?”	No sabe, dice “visita Agent Factory”	DB tiene <code>CustomAgent</code> con agentes reales
“¿Qué MCPs tengo conectados?”	No tiene idea	DB tiene <code>CustomMcp</code> con servidores reales
“¿Mi plan incluye turbo?”	Responde con turbo del plan	No consulta <code>turboEnabled</code> , <code>turboModel</code> , <code>turboProvider</code> del <code>User</code>

Causa raíz: `buildGlobalState()` en `lib/octopus-global-state.ts` consulta muchas tablas pero omite datos críticos de identidad y configuración del usuario.

2. ESTADO ACTUAL — Auditoría Completa

2.1 Lo que `buildGlobalState()` Sí consulta hoy:

- ✓ `User.planId`, `User.turboEnabled`, `User.name`
- ✓ `ArmConnection` (Brazos conectados + credentials para Ollama)
- ✓ `VoiceAgent` (todos los agents del usuario)
- ✓ `SalesAgent` + `SalesAgentLead` (agentes de venta + leads)
- ✓ `SmartDevice` (IoT count + online count)
- ✓ `Subscription` (status, days left, canceling)
- ✓ `GrowthLead` (count + 10 recientes)
- ✓ `Campaign` (lista completa con métricas)
- ✓ `GrowthAction` (pendientes count)
- ✓ `CreativeAsset` + `Project` (counts)
- ✓ `SocialPost` + `SocialConnection` (posts + plataformas)
- ✓ `Invoice` (total + drafts)
- ✓ `CalendarEvent` (upcoming count)
- ✓ `KnowledgeDocument` (count)
- ✓ `ApiKey` (total, active, service names)
- ✓ `SkillExecution` (grouped by skillId + count)

2.2 Lo que NO consulta (BRECHAS CRÍTICAS):

- | | |
|--|--|
| ✗ <code>User.businessEmail</code> | → OCTOPUS no sabe el email de negocio configurado |
| ✗ <code>User.email</code> | → Solo se pasa en <code>userIdentityPrompt</code> , no en <code>globalState</code> |
| ✗ <code>User.turboModel</code> | → No sabe qué modelo turbo usa el usuario |
| ✗ <code>User.turboProvider</code> | → No sabe qué proveedor turbo |
| ✗ <code>User.elevenLabsKey</code> | → No sabe si tiene ElevenLabs configurado |
| ✗ <code>User.elevenLabsEnabled</code> | → No sabe si está habilitado |
| ✗ <code>User.shellyCloudServer</code> | → No sabe si tiene Shelly IoT configurado |
| ✗ <code>User.planPeriod</code> | → No sabe si paga mensual o anual |
| ✗ <code>User.activeWorkspaceId</code> | → No sabe en qué workspace está |
| ✗ <code>User.showWatermark</code> | → No sabe si tiene watermark activo |
| ✗ <code>User.promoTrialEndsAt</code> | → No sabe si tiene trial activo |
| ✗ <code>User.brazosUnlimitedUntil</code> | → No sabe si tiene brazos ilimitados |
| ✗ <code>CustomAgent</code> (Agent Factory) | → NO consulta los agentes custom creados |
| ✗ <code>CustomSkill</code> (Skill Factory) | → NO consulta los skills custom creados |
| ✗ <code>CustomMcp</code> (MCP Factory) | → NO consulta los MCPs configurados |
| ✗ <code>Workspace</code> | → No sabe workspaces del usuario |
| ✗ <code>ConsciousnessState</code> | → Ironía: existe tabla de consciencia pero no se consulta |
| ✗ <code>HostedSite</code> | → No sabe qué sitios tiene publicados |
| ✗ <code>BookingConfig</code> | → No sabe si tiene booking configurado |
| ✗ <code>OnboardingData</code> | → No sabe qué respondió en onboarding |
| ✗ <code>TaskItem</code> | → No sabe las tareas pendientes del usuario |
| ✗ <code>BrowserTemplate</code> | → No sabe qué templates de browser tiene |
| ✗ <code>ScheduledBrowserTask</code> | → No sabe qué tareas programadas tiene |
| ✗ <code>NexusProject</code> | → No sabe sus proyectos en Nexus |

2.3 Cómo llega el estado al LLM (flujo actual):

```
chat/route.ts:
1. contextRouter(message) → detecta módulos activos
2. buildGlobalState(userId, activeModules) → consulta DB en 3 batches
3. fullSystemPrompt = [
    basePrompt,                // CORE_PERSONALITY (~3K tokens)
    userIdentityPrompt,        // Solo nombre + email de login
    globalStatePrompt,         // Estado global (variable ~500-2K tokens)
    memoryPrompt,              // Memoria semántica reciente
    ragContext.contextPrompt,   // RAG knowledge base
    systemContextPrompt,        // Contexto del módulo activo
    chainInstructions,          // Anti-hallucination rules
  ].join('\n')
```

Problema clave: `userIdentityPrompt` solo tiene `name` y `email` de login. NO tiene `businessEmail`, ni configuración del usuario, ni sus factories.

3. ARQUITECTURA PROPUESTA

3.1 Concepto: “Cerebro Consciente” (Brain Module)

No es RAG (estático). No es un skill (se ejecuta on-demand).

Es una **expansión de `buildGlobalState()`** que se ejecuta en CADA chat y le da al LLM conocimiento completo del usuario y la plataforma.

3.2 Principios de Diseño:

1. **Token Budget:** Máximo ~1500 tokens para el bloque de consciencia. Compacto pero completo.
2. **Zero Hallucination:** Solo datos de DB. Si un campo es null, se dice “no configurado”.
3. **Batched Queries:** Mantener el patrón de 3 batches para no saturar el pool de conexiones (max ~10).
4. **Module-Aware:** Secciones detalladas solo cuando el módulo está activo (ya implementado en Fase 3).
5. **Fail-Safe:** Si cualquier query falla, el sistema sigue funcionando con datos parciales.

3.3 Diagrama de Flujo:



4. IMPLEMENTACIÓN DETALLADA

4.1 ARCHIVO: `lib/octopus-global-state.ts`

4.1.1 Expandir el SELECT del User (BATCH 1)

Actual (línea 64-67):

```
prisma.user.findUnique({
  where: { id: userId },
  select: { planId: true, turboEnabled: true, name: true },
})
```

Propuesto:

```
prisma.user.findUnique({
  where: { id: userId },
  select: {
    name: true,
    email: true,
    planId: true,
    planPeriod: true,
    turboEnabled: true,
    turboModel: true,
    turboProvider: true,
    businessEmail: true,
    elevenLabsEnabled: true,
    elevenLabsKey: true, // Solo para saber si existe (boolean check)
    shellyCloudServer: true, // Solo para saber si existe (boolean check)
    showWatermark: true,
    activeWorkspaceId: true,
    promoTrialEndsAt: true,
    brazosUnlimitedUntil: true,
    onboardedAt: true,
    tourCompleted: true,
  },
})
```

4.1.2 Agregar nuevas queries al BATCH 1

Agregar junto a las queries existentes del batch 1:

```
// NEW: ConsciousnessState
prisma.consciousnessState.findUnique({
  where: { userId },
  select: { overallLevel: true, operativa: true, datos: true, predictiva: true, relacional: true },
}).catch(() => null),

// NEW: OnboardingData
prisma.onboardingData.findUnique({
  where: { userId },
  select: { projectName: true, projectType: true, objective: true },
}).catch(() => null),

// NEW: BookingConfig exists
prisma.bookingConfig.findFirst({
  where: { userId },
  select: { id: true },
}).catch(() => null),
```

 **NOTA:** Verificar que BATCH 1 no exceda ~10 queries paralelas. Si lo hace, mover algunas al BATCH 2.

4.1.3 Nuevo BATCH 2: Factories + Custom Entities

Reemplazar/expandir el batch 2 actual:

```
// =====
// BATCH 2 – Factories + Custom Entities
// =====
const [
  // Existing
  leadCount, recentLeads, campaignsRaw, pendingActionCount,
  assetCount, projectCount, socialPostCount, socialConnections,
  // NEW
  customAgents, customSkills, customMcps,
  hostedSites, pendingTasks, scheduledTasks, nexusProjects,
] = await withDbRetry(() => Promise.all([
  // ... existing queries ...

  // NEW: Custom Agents (Agent Factory)
  prisma.customAgent.findMany({
    where: { userId },
    select: { name: true, category: true, isActive: true, model: true, icon: true },
  }).catch(() => []),

  // NEW: Custom Skills (Skill Factory)
  prisma.customSkill.findMany({
    where: { userId },
    select: { name: true, category: true, isActive: true, usageCount: true },
  }).catch(() => []),

  // NEW: Custom MCPs (MCP Factory)
  prisma.customMcp.findMany({
    where: { userId },
    select: { name: true, isConnected: true, capabilities: true, icon: true },
  }).catch(() => []),

  // NEW: Hosted Sites
  prisma.hostedSite.findMany({
    where: { userId },
    select: { name: true, slug: true, customDomain: true, status: true },
    take: 5,
  }).catch(() => []),

  // NEW: Pending Tasks
  prisma.taskItem.count({
    where: { userId, status: 'pending' },
  }).catch(() => 0),

  // NEW: Scheduled Browser Tasks
  prisma.scheduledBrowserTask.count({
    where: { userId, isActive: true },
  }).catch(() => 0),

  // NEW: Nexus Projects
  prisma.nexusProject.count({
    where: { userId },
  }).catch(() => 0),
]), { label: 'GlobalState-B2' })
```

⚠ IMPORTANTE: Esto suma ~7 queries nuevas al batch 2. Batch 2 actual tiene 8. Total = 15.
Si el pool de conexiones no aguantaba 15 paralelas, dividir en BATCH 2A y BATCH 2B.

Pero dado que el max es ~10 conexiones y las queries son rápidas (count/findMany con take), 15 paralelas deberían estar OK porque se liberan rápido.

ALTERNATIVA SEGURA: Dividir en 4 batches en vez de 3:

- Batch 1: User + Identity (7 queries)
- Batch 2: Agents + Factories (7 queries)
- Batch 3: Growth + Social + Creative (8 queries)
- Batch 4: Invoices + Calendar + Misc (7 queries)

4.1.4 Expandir el interface `GlobalStateData`

Agregar:

```
export interface GlobalStateData {
  // EXISTING...

  // NEW: User Identity
  identity: {
    name: string | null
    email: string
    businessEmail: string | null
    planPeriod: string
    turboModel: string | null
    turboProvider: string | null
    hasElevenLabs: boolean
    hasShellyIoT: boolean
    showWatermark: boolean
    hasPromoTrial: boolean
    promoTrialDaysLeft: number
    hasBrazosUnlimited: boolean
    isOnboarded: boolean
  }

  // NEW: Factories
  customAgents: { name: string; category: string; active: boolean; model: string }[]
  customSkills: { name: string; category: string; active: boolean; usageCount:
number }[]
  customMcps: { name: string; connected: boolean; capabilities: string[] }[]

  // NEW: Platform Features
  hostedSites: { name: string; domain: string; published: boolean }[]
  pendingTasks: number
  scheduledBrowserTasks: number
  nexusProjects: number
  consciousness: { level: number; operativa: number; datos: number; predictiva:
number; relacional: number } | null

  // NEW: Business Context
  onboarding: { projectName: string; projectType: string; objective: string } | null
  hasBookingConfig: boolean
}
```

4.1.5 Nuevas Secciones del Prompt

Agregar estas secciones al builder de prompt:

```
// --- IDENTITY (ALWAYS – critical for self-awareness) ---
const identityParts: string[] = []
if (userRecord?.businessEmail) identityParts.push(`Email negocio: ${userRecord.businessEmail}`)
if (userRecord?.email) identityParts.push(`Email login: ${userRecord.email}`)
identityParts.push(`Plan: ${planLabel} (${userRecord?.planPeriod || 'monthly'})`)
if (userRecord?.turboEnabled) {
  identityParts.push(`Turbo: ☒ ${userRecord.turboModel || 'default'} via ${userRecord.turboProvider || 'openai'}`)
} else {
  identityParts.push(`Turbo: ☐`)
}
if (userRecord?.promoTrialEndsAt && new Date(userRecord.promoTrialEndsAt) > new Date()) {
  const trialDays = Math.ceil((new Date(userRecord.promoTrialEndsAt).getTime() - Date.now()) / 86400000)
  identityParts.push(`Trial: ${trialDays}d restantes`)
}
sections.push(`[IDENTIDAD] ${identityParts.join(' · ')}`)

// --- FACTORIES (ALWAYS – core capabilities awareness) ---
const factoryParts: string[] = []
const activeCustomAgents = customAgents.filter(a => a.isActive)
const activeCustomSkills = customSkills.filter(s => s.isActive)
const connectedMcps = customMcps.filter(m => m.isConnected)

factoryParts.push(`Agents: ${activeCustomAgents.length}/${customAgents.length}`)
if (activeCustomAgents.length > 0) {
  factoryParts.push(`[${activeCustomAgents.map(a => `${a.icon || '🤖'} ${a.name}`)].join(', ')]`)
}

factoryParts.push(`Skills: ${activeCustomSkills.length}/${customSkills.length}`)
if (activeCustomSkills.length > 0) {
  factoryParts.push(`[${activeCustomSkills.map(s => s.name).join(', ')]`)
}

factoryParts.push(`MCPs: ${connectedMcps.length}/${customMcps.length}`)
if (connectedMcps.length > 0) {
  factoryParts.push(`[${connectedMcps.map(m => m.name).join(', ')]`)
}

sections.push(`[FACTORIES] ${factoryParts.join(' · ')}`)

// --- PLATFORM FEATURES (compact) ---
const featureParts: string[] = []
if (hostedSites.length > 0) {
  featureParts.push(`Sites: ${hostedSites.filter(s => s.status === 'active').length} publicados`)
}
if (pendingTasks > 0) featureParts.push(`Tasks: ${pendingTasks} pendientes`)
if (scheduledTasks > 0) featureParts.push(`Browser auto: ${scheduledTasks} activas`)
if (nexusProjects > 0) featureParts.push(`Nexus: ${nexusProjects} proyectos`)
if (userRecord?.elevenLabsKey) featureParts.push(`ElevenLabs: ☒`)
if (userRecord?.shellyCloudServer) featureParts.push(`Shelly IoT: ☒`)
if (featureParts.length > 0) {
  sections.push(`[PLATAFORMA] ${featureParts.join(' · ')}`)
}

// --- CONSCIOUSNESS (when diagnostic active) ---
if (isActive('diagnostic') && consciousnessData) {
  sections.push(`[🧠 CONSCIENCIA] Nivel: ${consciousnessData.overallLevel}% | Op:${con`

```



```
consciousnessData.operativa} Dat:${consciousnessData.datos} Pred:${consciousness-
Data.predictiva} Rel:${consciousnessData.relacional}`)
}
```

4.2 ARCHIVO: lib/octopus-personality.ts

4.2.1 Actualizar la sección “CONCIENCIA TOTAL”

Reemplazar el bloque actual (líneas ~127-133) con:

```
## 🦑 CONCIENCIA TOTAL 📄 TU IDENTIDAD Y ESTADO REAL
Recibes bloques de estado: [IDENTIDAD], [PLAN], [BRAZOS], [FACTORIES], [PLATAFORMA], [
VOICE AGENTS], [SALES AGENTS], [IoT], [GROWTH ENGINE], [CREATIVO], [SOCIAL BRIDGE], [F
ACTURAS], [CALENDARIO], [KB], [API HUB], [SUSCRIPCIÓN], [🦑 CONSCIENCIA], [💡 RECOMEN-
DACIONES].

**REGLAS ABSOLUTAS DE CONSCIENCIA**:
1. **IDENTIDAD**: Tu email de negocio está en [IDENTIDAD]. Si dice "Email negocio:
X", ESO es el email del negocio. El email de login es diferente. NUNCA confundas
email de login con email de negocio.
2. **FACTORIES**: [FACTORIES] muestra tus agentes, skills y MCPs REALES. Si dice "Agen
ts: 0/0" 📄 el usuario tiene 0 agentes. NO inventes que tiene agentes.
3. **USA LOS DATOS REALES** del estado global. Si dice "[VOICE AGENTS] 0", responde "n
o tienes Voice Agents configurados". NUNCA inventes datos.
4. **BRAZOS**: Si un brazo NO aparece en [BRAZOS] como ✅, NO está conectado. No puede
s usarlo. Di honestamente qué falta.
5. **RESPONDE INLINE** 📄 cuando pregunten "¿qué tengo en X?", responde CON LOS DATOS d
el estado. NUNCA respondas con navigate o links cuando piden información.
6. **HONESTIDAD sobre lo que NO ves**: Algunas cosas viven en localStorage del nave-
gador. Si no aparecen en tu estado, di: "No tengo visibilidad de eso desde aquí.
Verifica en /dashboard/X".
7. **Si el dato es null/vacío**: Di "no configurado" o "no tienes X todavía", NUNCA in
ventes un valor.
```

4.2.2 Eliminar la lista hardcodeda de BUILT_IN_SKILLS

En octopus-global-state.ts línea 228:

```
// ELIMINAR ESTO:
const BUILT_IN_SKILLS = ['image-skill', 'game-skill', 'code-refiner', 'wildverse-
seo', 'content-publisher', 'lead-to-asset']
```

Los skills reales ahora vienen de CustomSkill en la DB. El concepto de “built-in skills” era un hardcoded falso.

4.3 ARCHIVO: app/api/jarvis/chat/route.ts

4.3.1 Eliminar userIdentityPrompt redundante

El userIdentityPrompt actual (línea 236-238) es redundante porque la nueva sección [IDENTIDAD] del globalState ya incluye nombre, email y businessEmail.

Opción A: Eliminarlo completamente (recomendado)

Opción B: Dejarlo como fallback en caso de que globalState falle

Recomiendo **Opción B** — dejarlo como fallback pero que globalState sea la fuente primaria:

```
// userIdentityPrompt solo como fallback si globalState falla
const userIdentityPrompt = globalStatePrompt
? '' // globalState ya incluye [IDENTIDAD]
: (session.user.name
  ? `\n[USUARIO] ${session.user.name} (${session.user.email || '?'})\n`
  : '')
```

5. PROMPT ENGINEERING PARA CONSCIENCIA

5.1 Formato del bloque de consciencia (ejemplo real)

```
[IDENTIDAD] Email negocio: contact@octopuskills.com • Email login: 1billion-
topview@gmail.com • Plan: 🏢 Business (monthly) • Turbo: ✅ gpt-4.1 via openai
[BRAZOS] ✅ Google Workspace • ✅ Telegram
[CAPS] Email/Calendar/Drive • Telegram • Growth(47 leads) • Sales(2) • Voice(1)
[FACTORIES] Agents: 2/3 [🤖 SEO Writer, 🤖 Email Drafter] • Skills: 4/5 [web-
scraper, seo-analyzer, lead-scorer, content-gen] • MCPs: 1/2 [Notion API]
[PLATAFORMA] Sites: 1 publicado • Tasks: 3 pendientes • ElevenLabs: ✅
[GROWTH ENGINE] 47 leads • 2 activas, 1 draft • 5 pendientes
[🧠 CONSCIENCIA] Nivel: 78% | Op:82 Dat:71 Pred:75 Rel:84
[💡] 📧 5 acciones pendientes | 📄 2 facturas en borrador
```

Token estimate: ~200-400 tokens dependiendo de cuántas factories/features tenga el usuario.

5.2 Reglas de compactación

- **Secciones vacías NO se emiten:** Si 0 agents, 0 skills, 0 MCPs → [FACTORIES] Agents: 0 • Skills: 0 • MCPs: 0 (una línea)
- **Listas se truncan a 5:** Si tiene 20 skills, solo muestra los 5 más usados
- **Detalles solo por módulo:** Campaign list solo cuando module=growth_engine activo
- **Boolean como emoji:** ✅❌ en vez de true/false

5.3 Jerarquía de inyección

Mantener el orden actual pero con la identidad más prominente:

```
const fullSystemPrompt = [
  basePrompt,           // CORE_PERSONALITY (identidad OCTOPUS)
  globalStatePrompt,    // 🧠 CONSCIOUSNESS (identidad USUARIO + todo el estado)
  memoryPrompt,         // Memoria semántica
  ragContext,           // Knowledge base
  systemContextPrompt,  // Contexto del módulo
  chainInstructions,     // Anti-hallucination
].join('\n')
```

Nota: `userIdentityPrompt` se elimina o queda como fallback vacío.

6. ANTI-ALUCINACIÓN REFORZADA

6.1 Reglas específicas de consciencia

Agregar al `chainInstructions` o al `CORE_PERSONALITY` :

🚨 REGLAS DE CONSCIENCIA (PRIORIDAD MÁXIMA)

1. ****DATO REAL vs DATO INVENTADO****: Si un dato aparece en tu estado [IDENTIDAD], [BRAZOS], [FACTORIES], etc. ➡ es **REAL**. Si **NO** aparece ➡ **NO** existe. NUNCA supongas.

2. ****EMAIL DE NEGOCIO****: El campo "Email negocio" en [IDENTIDAD] es el businessEmail configurado en Settings. Es DIFERENTE del email de login. Si te preguntan "mi email de negocio" o "email de contacto", responde con el businessEmail. Si es **null**, di "no tienes email de negocio configurado. Puedes configurarlo en Settings".

3. ****CAPABILITIES REALES****: Solo puedes **usar**:

- Brazos que aparezcan como  en [BRAZOS]
- Skills que aparezcan en [FACTORIES]
- Agents que aparezcan en [FACTORIES]
- MCPs que aparezcan como conectados en [FACTORIES]

Si algo **NO** aparece ➡ **NO** lo tienes. Di "para usar X, primero conecta Y".

4. ****CONTEOS****: Si dice "Skills: 4/5" ➡ 4 activos de 5 totales. Si te preguntan "cuántos skills tengo" ➡ responde "4 activos, 5 en total".

5. ****NUNCA INVENTES NOMBRES****: Si [FACTORIES] lista skills por nombre, **SOLO** menciona esos nombres. **No** inventes skills que **no** existen.

6. ****PLAN Y LÍMITES****: Si dice "Plan: Starter" ➡ el usuario está en Starter. **No** le digas que tiene features de Business. Conoces los límites de cada **plan**.

7. ****TURBO****: Si dice "Turbo: ✖" ➡ el usuario **NO** tiene turbo. **No** le digas que puede usar modelos turbo **sin** configurarlo primero.

6.2 Test cases para validación

#	Input	Expected Response	Validates
1	"¿Cuál es mi email de negocio?"	businessEmail de [IDENTIDAD] o "no configurado"	Identity awareness
2	"¿Cuántos skills tengo?"	Count real de [FACTORIES]	Factory awareness
3	"¿Qué brazos tengo conectados?"	Lista de [BRAZOS] 	Brazo awareness
4	"Envíame un email" (sin Gmail conectado)	"No tienes Gmail conectado"	Capability honesty
5	"¿Cuántos agentes he creado?"	Count de [FACTORIES] Agents	Agent Factory awareness
6	"¿Tengo turbo activado?"	Estado de turbo + modelo/provider	Config awareness
7	"¿Qué MCPs tengo?"	Lista de [FACTORIES] MCPs	MCP awareness
8	"Dame un resumen de mi cuenta"	Usa TODOS los bloques de estado	Full consciousness
9	"¿Cuántos leads tengo?"	Count real de [GROWTH ENGINE]	Data accuracy
10	"¿Mi plan incluye voice agents?"	Basado en planId + limits	Plan awareness

7. TESTING & VALIDACIÓN

7.1 Test de rendimiento

1. Medir tiempo de buildGlobalState() antes y después
 - Target: < 500ms con 4 batches
 - Actual: ☐ 200-300ms con 3 batches
 - Aceptable: hasta 600ms con 4 batches
2. Medir **tokens** del prompt de consciencia
 - Target: < 500 **tokens** para usuario típico
 - Max: 1500 **tokens** para usuario con **todo** configurado
3. Verificar que NO se rompa el pool de conexiones
 - Cada batch max 8-10 queries paralelas
 - **Total**: 4 batches ☒ 8 = ☐ 32 queries secuencial-paralelas
 - Test: chatear rápido 5 veces seguidas y verificar que no hay timeouts

7.2 Test funcional (los 10 del punto 6.2)

Ejecutar los 10 tests en chat real después de implementar.

Cada test debe comparar la respuesta de OCTOPUS con los datos reales de la DB.

7.3 Regression tests

Verificar que funcionalidades existentes NO se rompen:

- Growth Engine prospecting sigue funcionando
 - Creative generation sigue funcionando
 - Gmail/Calendar sigue funcionando
 - Voice agents sigue funcionando
 - No hay errores de TypeScript en build
-

8. ARCHIVOS A MODIFICAR

Resumen de cambios por archivo:

Archivo	Cambios	Impacto
<code>lib/octopus-global-state.ts</code>	Expandir User select, agregar queries de Factories/Features/Identity, expandir interface, agregar secciones al prompt	PRINCIPAL — El 80% del trabajo
<code>lib/octopus-personality.ts</code>	Actualizar sección “CONCIENCIA TOTAL” con nuevas reglas, eliminar hardcoded BUILT_IN_SKILLS reference	SECUNDARIO — Reglas de comportamiento
<code>app/api/jarvis/chat/route.ts</code>	Eliminar/reducir userIdentityPrompt redundante	MENOR — Cleanup
<code>prisma/schema.prisma</code>	NO TOCAR — ya tiene todos los modelos necesarios	NINGUNO

Orden de implementación recomendado:

1. **Paso 1:** Expandir `GlobalStateData` interface con nuevos campos
2. **Paso 2:** Agregar nuevas queries en batches (reorganizar si necesario)
3. **Paso 3:** Procesar los datos nuevos en la sección “PROCESS DATA”
4. **Paso 4:** Agregar secciones [IDENTIDAD], [FACTORIES], [PLATAFORMA] al builder de prompt
5. **Paso 5:** Actualizar `octopus-personality.ts` con las reglas de consciencia
6. **Paso 6:** Limpiar `chat/route.ts` (userIdentityPrompt)
7. **Paso 7:** Build + Test
8. **Paso 8:** Ejecutar los 10 test cases en chat real
9. **Paso 9:** Deploy

Riesgos y mitigaciones:

Riesgo	Probabilidad	Mitigación
Pool de conexiones saturado	Media	Dividir en 4 batches, max 8 queries/batch
Latencia aumentada	Baja	Queries son simples counts/findMany con take
Token budget excedido	Baja	Compactación agresiva, secciones vacías omitidas
Campos no existen en schema	Nula	Verificado — todos los modelos/campos existen
TypeScript errors	Baja	Todos los campos tienen tipos definidos en schema

 RESUMEN EJECUTIVO

Estado actual: OCTOPUS consulta ~22 queries en 3 batches pero le falta identidad del usuario (businessEmail, turbo config), factories (agents, skills, MCPs custom) y features de plataforma (hosted sites, tasks, consciousness level).

Solución: Expandir buildGlobalState() con ~12 queries adicionales divididas en 4 batches. Agregar 3 secciones nuevas al prompt: [IDENTIDAD], [FACTORIES], [PLATAFORMA]. Actualizar reglas de personalidad con anti-alucinación específica para datos de identidad.

Esfuerzo estimado: 1 archivo principal (octopus-global-state.ts), 1 archivo secundario (octopus-personality.ts), 1 archivo menor (chat/route.ts). ~200-300 líneas de código nuevo/modificado.

Resultado esperado: OCTOPUS sabrá responder correctamente “¿cuál es mi email de negocio?”, “¿cuántos skills tengo?”, “¿qué agentes he creado?”, “¿tengo turbo?”, etc. — con datos REALES de la base de datos, no inventados.